

PROGRAMMING / SYNTAX / THINKING



CODE, FROM ZERO

A clean, practical guide to learning the language of computers

Scott Brodie Forsyth

First edition / 2026

A note to the reader

Programming can look like a wall of strange punctuation. It is not. Every program is built from a small number of ideas: values, names, decisions, repetition, functions, data structures, and communication between parts of a system. The difficult part is not memorising every symbol. The difficult part is learning how to combine simple pieces without losing track of what the computer is doing.

This book starts with no assumed knowledge. It uses Python as the first language because its syntax is compact and readable, then compares the same ideas in JavaScript, TypeScript, SQL, HTML, CSS, C++, Java, Bash, and common data tools. The goal is not to pretend that one book can contain every library or every specialism. The goal is to give you a durable map of the field, enough syntax to read and write real code, and a path for going deeper.

Core idea. This is a book about the grammar of code and the ideas behind it. Read the examples actively: predict the output, type the code yourself, change one thing, and observe what breaks.

How to use this book

Each chapter has four layers:

1. **The idea:** what problem a feature solves.
2. **The syntax:** the shapes you need to write.
3. **The mental model:** what the computer is doing.
4. **Practice:** small tasks that turn recognition into skill.

Do not wait until you feel ready to code. A useful loop is:

Read a little → **Type it** → **Predict** → **Run** → **Change it** →
Explain it

When an example uses a terminal, the leading `\$` is a prompt, not part of the command. When an example uses Python, save it in a file ending in `.py`, or paste it into a Python interpreter.

The map of programming

Area	What you build or study
Programming languages	Syntax, types, functions, objects, compilers, interpreters
Algorithms and data structures	Ways to store information and solve problems efficiently
Web development	Pages, browsers, servers, APIs, databases, authentication
Systems	Operating systems, memory, processes, networks, hardware
Data and AI	Tables, statistics, numerical computing, machine learning
Software engineering	Testing, version control, architecture, teamwork, deployment
Security	Threats, permissions, safe input handling, cryptography, privacy

The first half of this book gives you the shared foundation. The second half shows how that foundation branches into the fields above.

Contents

A note to the reader	i
How to use this book	ii
The map of programming	iii
I The core language	1
1 What programming actually is	2
1.1 Instructions, data, and state	2
1.2 Syntax and semantics	2
1.3 How code runs	2
1.4 Your first program	3
1.5 A first debugging habit	3
2 Values, variables, and types	4
2.1 Values are the pieces of information	4
2.2 Names and assignment	4
2.3 Converting between types	5
2.4 Checking a type	5
3 Expressions and operators	6
3.1 Expressions produce values	6
3.2 Arithmetic	6
3.3 Comparison and Boolean logic	6
3.4 Truthiness	7
4 Strings, input, and output	8
4.1 Strings are sequences of characters	8
4.2 Formatting text	8
4.3 Useful string methods	8
4.4 Input	9
5 Collections: lists, tuples, sets, and dictionaries	10
5.1 Lists	10
5.2 Tuples and sets	10
5.3 Dictionaries	10
5.4 Comprehensions	11
6 Control flow	12
6.1 Conditions	12
6.2 For loops	12
6.3 While loops	12
6.4 Break, continue, and enumerate	13

6.5	Matching patterns	13
7	Functions, scope, and modules	14
7.1	Functions name a process	14
7.2	Parameters and arguments	14
7.3	Flexible arguments	14
7.4	Scope	15
7.5	Modules and imports	15
8	Errors, debugging, and testing	16
8.1	Three kinds of errors	16
8.2	Handling exceptions	16
8.3	Raising errors	16
8.4	Assertions and tests	17
8.5	A practical debugging loop	17
9	Files, paths, and data formats	18
9.1	Reading and writing text	18
9.2	JSON	18
9.3	CSV	18
9.4	Paths and directories	19
10	Objects and classes	20
10.1	Objects bundle data and behaviour	20
10.2	Encapsulation and invariants	20
10.3	Inheritance and composition	20
10.4	Dataclasses	21
10.5	Type hints	21
11	Data structures and algorithms	22
11.1	Choosing a structure	22
11.2	Complexity intuition	22
11.3	Searching and sorting	22
11.4	Recursion	23
11.5	Graphs	23
12	The terminal, environments, and Git	24
12.1	The command line	24
12.2	Virtual environments	24
12.3	Git’s mental model	24
II	The programming landscape	26
13	HTML: the structure of the web	27
13.1	Documents and elements	27
13.2	Forms	27
13.3	Accessibility basics	27
14	CSS: styling and layout	28
14.1	Selectors and declarations	28
14.2	The box model	28
14.3	Flexbox and grid	28

15 JavaScript: behaviour in the browser	30
15.1 Variables and values	30
15.2 Functions and conditions	30
15.3 Arrays and objects	30
15.4 The DOM and events	31
16 TypeScript: JavaScript with a type system	32
16.1 Annotations and inference	32
16.2 Objects, interfaces, and unions	32
16.3 Generics	32
16.4 Narrowing	33
17 SQL and databases	34
17.1 Tables and relationships	34
17.2 Selecting data	34
17.3 Inserting and updating	34
17.4 Joins and aggregation	35
17.5 Parameterized queries	35
18 HTTP, APIs, JSON, and asynchronous code	36
18.1 Requests and responses	36
18.2 A JSON API client	36
18.3 Async and await	36
18.4 Promises in JavaScript	37
19 Backend programming	38
19.1 The server's job	38
19.2 Layers	38
19.3 Authentication and authorization	38
19.4 Validation	38
20 C and C++: closer to the machine	39
20.1 A small C++ program	39
20.2 References and pointers	39
20.3 Vectors, maps, and loops	39
20.4 Memory as a model	40
21 Java and the JVM	41
21.1 Classes and a main method	41
21.2 Types and collections	41
21.3 Exceptions and interfaces	41
III Beyond syntax	42
22 Programming paradigms and architecture	43
22.1 Imperative and declarative code	43
22.2 Procedural programming	43
22.3 Object-oriented programming	43
22.4 Functional programming	43
22.5 Architecture as boundaries	43
23 Concurrency, parallelism, and processes	44

23.1	Concurrency is not the same as parallelism	44
23.2	Processes and threads	44
23.3	Shared state and locks	44
24	Security and defensive programming	46
24.1	Threat modelling	46
24.2	The boundary rule	46
24.3	Secrets	46
24.4	Common failures	46
25	Performance, deployment, and cloud systems	47
25.1	Measure before optimising	47
25.2	Caching	47
25.3	Build and deployment	47
25.4	Containers	47
25.5	Observability	48
26	Data science and machine learning syntax	49
26.1	Arrays and tables	49
26.2	A minimal learning loop	49
26.3	Data leakage and evaluation	49
27	How computers and networks work	50
27.1	Bits, bytes, and memory	50
27.2	Call stacks and heaps	50
27.3	Networks	50
27.4	The request path	50
28	Projects and a path from zero to capable	51
28.1	The smallest useful projects	51
28.2	A project checklist	51
28.3	A twelve-week route	52
A	Syntax atlas	53
A.1	Python	53
A.2	JavaScript and TypeScript	53
A.3	SQL	53
A.4	HTML, CSS, and Bash	54
B	Glossary	55
C	Selected practice solutions	57
C.1	A number range	57
C.2	Word lengths	57
C.3	A small command-line to-do model	57
	Final checklist	58



The core language

What programming actually is

1.1 Instructions, data, and state

A program is a precise description of a process. It receives or creates data, transforms that data, and produces an effect: text on a screen, a saved file, a database update, a moving character, or a response sent over a network.

The word **state** means the information that is true at a particular moment. If a game has a score of 12 and the player has three lives, that is part of the game's state. A line of code changes state by assigning a new value, adding an item, calling a function, or triggering an external action.

Core idea. At the deepest level, programming is controlled change. You describe which state exists now, which operation should happen next, and what state should exist afterward.

1.2 Syntax and semantics

Syntax is the grammar of a language: where parentheses, colons, quotes, commas, and indentation are allowed. **Semantics** is the meaning of a valid program. Two pieces of code can have different syntax but the same meaning.

For example, these Python statements have equivalent results:

```
1 total = 2 + 3
```

```
1 total = sum([2, 3])
```

The first is more direct. The second uses a function that can also sum a longer collection. Learning syntax is useful, but syntax is only the surface. The important question is always: what value does this expression produce, and what state does this statement change?

1.3 How code runs

There are three common ways a language reaches the computer:

- An **interpreter** reads and executes code, often statement by statement.
- A **compiler** translates source code into another form before it runs, often machine code.
- A **virtual machine** runs an intermediate representation, such as Java bytecode.

The boundaries are not absolute. Modern languages often combine these approaches. Python first turns source into bytecode and then runs it on the Python virtual machine. JavaScript engines interpret, compile, optimise, and de-optimise code while a program is running.

1.4 Your first program

```
1 print("Hello, world!")
```

The name `print` refers to a function. The parentheses contain an argument. The text between quotation marks is a string literal. Running this statement produces visible output, but it does not change a variable in your program.

```
1 name = "Ada"
2 print("Hello, " + name + "!")
```

The second example stores a value under the name `name`, combines three strings with `+`, and sends the result to `print`.

Try it. Change the name to your own name. Then add a second variable called `year` and print a sentence containing both values. Before running it, predict the exact output.

1.5 A first debugging habit

When code does not work, do not stare at the whole program. Reduce the problem. Ask:

1. What did I expect to happen?
2. What happened instead?
3. Which line first produces the wrong value?
4. What is the smallest test that isolates that line?

This habit scales from a five-line exercise to a large production service.

Values, variables, and types

2.1 Values are the pieces of information

A **value** is a piece of data. Common values include numbers, text, true-or-false values, collections, and the special value meaning "nothing." A **type** describes what kind of value something is and which operations make sense for it.

```
1 age = 26           # int: a whole number
2 height = 1.96      # float: a decimal number
3 name = "Scott"     # str: text
4 student = True     # bool: True or False
5 missing = None     # NoneType: no value
```

Python lets a name point to values of different types at different times:

```
1 thing = 10
2 thing = "ten"
```

This is called dynamic typing. It is flexible, but it means you must pay attention to what a name currently refers to. Other languages, such as Java and TypeScript, can require a variable to have a declared type.

2.2 Names and assignment

The equals sign in an assignment is not a mathematical claim that both sides are permanently equal. It means: evaluate the expression on the right, then bind the result to the name on the left.

```
1 score = 10
2 score = score + 5
```

The second line reads the old value 10, calculates 15, and then stores 15 under the same name. A shorter spelling is:

```
1 score += 5
```

Good names communicate intent. Prefer `days_remaining` to `x` when the meaning matters. In Python, variables usually use `snake_case`; constants are often written in `UPPER_CASE`.

2.3 Converting between types

Input from a user arrives as text. Convert it before doing numeric work.

```
1 age_text = "26"
2 age = int(age_text)
3 height = float("1.96")
4 label = str(2026)
```

Not every conversion is valid. `int("hello")` raises an exception because the text does not describe a whole number.

2.4 Checking a type

Use `type` while learning or debugging. In ordinary application code, prefer asking whether a value supports the operation you need rather than building a long chain of type checks.

```
1 value = 42
2 print(type(value))
3 print(isinstance(value, int))
```

Remember. A variable is a name, not a box with a permanent type. Assignment makes the name refer to a value. The value has a type, and the type determines valid operations.

Expressions and operators

3.1 Expressions produce values

An **expression** is code that can be evaluated to a value. A literal such as 7 is an expression. So are `2 + 3`, a function call, and a variable name.

```
1 price = 12.50
2 quantity = 3
3 subtotal = price * quantity
```

An expression can be nested inside another expression:

```
1 total = (price * quantity) * 1.25
```

3.2 Arithmetic

Operator	Meaning
+	addition
-	subtraction
*	multiplication
/	division, producing a float in Python
//	floor division
\%	remainder, or modulo
**	exponentiation

```
1 print(7 / 2)      # 3.5
2 print(7 // 2)     # 3
3 print(7 % 2)      # 1
4 print(2 ** 3)     # 8
```

Use parentheses when they make precedence clear. Even when you know the operator rules, explicit grouping helps the next reader.

3.3 Comparison and Boolean logic

Comparisons produce Boolean values:

```
1 age = 26
2 print(age >= 18)  # True
```

```
3 print(age == 26)      # True
4 print(age != 30)      # True
```

The single equals sign assigns. The double equals sign compares. Mixing them up is one of the first common errors.

Combine conditions with `and`, `or`, and `not`:

```
1 has_ticket = True
2 is_old_enough = True
3 allowed = has_ticket and is_old_enough
```

Python uses **short-circuit evaluation**: in `a and b`, if `a` is false, Python does not need to evaluate `b`; in `a or b`, a true `a` is enough.

3.4 Truthiness

Some values behave like false in a condition: `False`, `None`, zero, and empty collections. Most other values behave like true.

```
1 items = []
2 if not items:
3     print("The list is empty")
```

Use this feature when it makes code clearer. If the distinction between "empty" and "missing" matters, compare explicitly with `is None` or `== []`.

Try it. Write an expression that is true only when a number is between 10 and 20 inclusive. Test it with 9, 10, 15, 20, and 21.

Strings, input, and output

4.1 Strings are sequences of characters

Strings can use single or double quotes. Use the style that makes the inside text easiest to read.

```
1 single = 'hello'
2 double = "hello"
3 quote = "She said, 'run the code.'"
```

Special characters use escape sequences:

```
1 line = "first line\nsecond line"
2 path = "C:\\Users\\Ada"
```

4.2 Formatting text

F-strings are the most readable general-purpose formatting tool in Python.

```
1 name = "Ada"
2 score = 97.5
3 message = f"{name} scored {score:.1f}%"
4 print(message)
```

The expression inside braces is evaluated. The format specifier `:.1f` means one digit after the decimal point.

4.3 Useful string methods

```
1 text = "  Learn Python  "
2 clean = text.strip()
3 lower = clean.lower()
4 words = clean.split()
5 joined = "-".join(words)
6 print(clean, lower, words, joined)
```

Methods return new strings because strings are immutable: an existing string is not changed in place.

4.4 Input

```
1 name = input("What is your name? ")
2 age = int(input("How old are you? "))
3 print(f"Hello {name}. Next year you will be {age + 1}.")
```

Treat user input as untrusted text. It may be empty, malformed, unexpectedly long, or deliberately hostile. Validation belongs between receiving input and using it.

```
1 raw = input("Enter a positive whole number: ")
2 if raw.isdigit() and int(raw) > 0:
3     number = int(raw)
4     print(number * 2)
5 else:
6     print("That was not a positive whole number.")
```

Collections: lists, tuples, sets, and dictionaries

5.1 Lists

A list is an ordered, mutable collection. Indexing starts at zero.

```
1 tools = ["editor", "terminal", "browser"]
2 print(tools[0])
3 print(tools[-1])
4 tools.append("interpreter")
5 tools[1] = "git"
```

Slicing makes a new list. The end index is excluded.

```
1 numbers = [0, 1, 2, 3, 4, 5]
2 print(numbers[1:4]) # [1, 2, 3]
3 print(numbers[:3]) # [0, 1, 2]
4 print(numbers[3:]) # [3, 4, 5]
```

5.2 Tuples and sets

A tuple is an ordered collection that is commonly used for a fixed group of values. A set stores unique values and is useful for membership tests.

```
1 point = (3, 4)
2 x, y = point
3
4 tags = {"python", "code", "python"}
5 print(tags) # duplicate removed
6 print("code" in tags) # True
```

Sets do not promise a meaningful order. Do not rely on the order in which a set prints.

5.3 Dictionaries

A dictionary maps keys to values. It is the natural representation for a record with named fields.

```
1 user = {
2     "name": "Ada",
```

```
3     "age": 36,  
4     "skills": ["math", "programming"],  
5 }  
6  
7 print(user["name"])  
8 user["age"] += 1  
9 user["active"] = True
```

Use `get` when a key may be absent:

```
1 country = user.get("country", "unknown")
```

5.4 Comprehensions

A comprehension creates a collection from another iterable with an optional condition.

```
1 squares = [n * n for n in range(6)]  
2 even_squares = [n * n for n in range(10) if n % 2 == 0]  
3 lengths = {word: len(word) for word in ["code", "data"]}
```

Comprehensions are concise, not magical. If a comprehension becomes difficult to read, write an ordinary loop.

Try it. Create a list of the lengths of all words longer than four characters in a sentence. Then create a dictionary mapping each word to its length.

Control flow

6.1 Conditions

An `if` statement chooses which block should run. Python uses indentation to mark a block.

```
1 temperature = 18
2
3 if temperature < 0:
4     advice = "Wear a heavy coat."
5 elif temperature < 15:
6     advice = "Wear a jacket."
7 else:
8     advice = "A light layer should be fine."
9
10 print(advice)
```

The colon begins a block. Every line in the block must be indented consistently. Four spaces is the usual convention.

6.2 For loops

Use a `for` loop to visit each item in an iterable.

```
1 names = ["Ada", "Grace", "Alan"]
2 for name in names:
3     print(f"Hello, {name}!")
```

Use `range` when you need a sequence of integers. The stop value is excluded.

```
1 for number in range(1, 6):
2     print(number)
```

6.3 While loops

A `while` loop repeats while a condition remains true. You must change something that can eventually make the condition false.

```
1 attempts = 0
2 while attempts < 3:
3     print("Trying...")
4     attempts += 1
```

An accidental infinite loop is usually caused by forgetting the update or by updating the wrong variable.

6.4 Break, continue, and enumerate

```
1 for index, name in enumerate(names, start=1):
2     if name == "Grace":
3         continue
4     print(index, name)
```

`continue` skips to the next iteration. `break` exits the loop. Use both sparingly: a clear condition is often easier to understand than deeply nested control flow.

6.5 Matching patterns

Modern Python includes `match` for selecting among structural patterns.

```
1 command = ("move", 3)
2
3 match command:
4     case ("move", distance):
5         print(f"Move {distance} spaces")
6     case ("stop",):
7         print("Stop")
8     case _:
9         print("Unknown command")
```

The underscore is a catch-all pattern. An ordinary `if/elif` chain is still perfectly appropriate for simple conditions.

Functions, scope, and modules

7.1 Functions name a process

A function packages a process behind a name. It accepts inputs and may return an output.

```
1 def greet(name):  
2     return f"Hello, {name}!"  
3  
4 message = greet("Ada")  
5 print(message)
```

`return` immediately leaves the function. A function with no explicit return gives back `None`.

7.2 Parameters and arguments

Parameters are names in the function definition. Arguments are the values supplied at the call site.

```
1 def total_price(price, quantity=1, tax_rate=0.25):  
2     subtotal = price * quantity  
3     return subtotal * (1 + tax_rate)  
4  
5 print(total_price(10, quantity=3))
```

Keyword arguments improve clarity. Avoid mutable default arguments such as `items=[]`; that list would be shared between calls. Use `None` and create a new list inside instead.

7.3 Flexible arguments

```
1 def maximum(*numbers):  
2     if not numbers:  
3         raise ValueError("at least one number is required")  
4     return max(numbers)  
5  
6 settings = {"sep": " | ", "end": "\n"}  
7 print("a", "b", **settings)
```

The star gathers extra positional arguments into a tuple. Two stars gather extra keyword arguments into a dictionary.

7.4 Scope

A local variable exists inside the function that creates it. A global variable exists at module level. Prefer passing values into functions and returning values out. This makes the function easier to test and prevents hidden dependencies.

```
1 def add_tax(price, rate):
2     return price * (1 + rate)
3
4 price = add_tax(100, 0.25)
```

7.5 Modules and imports

Every Python file is a module. Import a module to reuse its definitions.

```
1 import math
2 from pathlib import Path
3
4 print(math.sqrt(25))
5 print(Path("notes.txt").suffix)
```

When a file is run directly, `__name__` is `"__main__"`. This pattern lets a file be both imported and run as a script.

```
1 def main():
2     print("Program started")
3
4 if __name__ == "__main__":
5     main()
```

Errors, debugging, and testing

8.1 Three kinds of errors

- A **syntax error** means the code is not valid in the language.
- A **runtime error** happens while valid code is running, such as dividing by zero.
- A **logic error** produces the wrong result without crashing.

The traceback is a map of the calls that led to the error. Read the last line first for the error type and message, then move upward to find your own code.

8.2 Handling exceptions

```
1 try:
2     number = int(input("Number: "))
3     result = 100 / number
4 except ValueError:
5     print("Please enter a whole number.")
6 except ZeroDivisionError:
7     print("The number cannot be zero.")
8 else:
9     print(result)
10 finally:
11     print("This always runs.")
```

Catch specific exceptions. A bare `except:` can hide programming defects and make debugging harder.

8.3 Raising errors

```
1 def withdraw(balance, amount):
2     if amount < 0:
3         raise ValueError("amount cannot be negative")
4     if amount > balance:
5         raise ValueError("insufficient balance")
6     return balance - amount
```

An exception is a controlled way to say that a function cannot fulfil its contract. The caller can decide how to present or recover from the failure.

8.4 Assertions and tests

An assertion documents an assumption during development.

```
1 def double(number):  
2     return number * 2  
3  
4 assert double(4) == 8  
5 assert double(-2) == -4
```

A proper test names the behaviour being checked:

```
1 def test_double_handles_zero():  
2     assert double(0) == 0  
3  
4 def test_withdraw_reduces_balance():  
5     assert withdraw(100, 25) == 75
```

Whether you use Python's built-in `unittest` or a tool such as `pytest`, tests are executable examples of intended behaviour.

8.5 A practical debugging loop

Write a small input that demonstrates the problem. Print or inspect intermediate values. Check types and lengths. Replace a large expression with named steps. Then keep the smallest failing example as a regression test.

Files, paths, and data formats

9.1 Reading and writing text

Use a context manager so the file is closed even if an error occurs.

```
1 from pathlib import Path
2
3 path = Path("notes.txt")
4 path.write_text("First note\nSecond note\n", encoding="utf-8")
5 text = path.read_text(encoding="utf-8")
6 print(text)
```

For larger files, process one line at a time:

```
1 with open("notes.txt", encoding="utf-8") as file:
2     for line in file:
3         print(line.strip())
```

9.2 JSON

JSON represents objects, arrays, strings, numbers, booleans, and null. It is common at API boundaries.

```
1 import json
2
3 record = {"name": "Ada", "active": True, "scores": [10, 12]}
4 text = json.dumps(record, indent=2)
5 loaded = json.loads(text)
6 print(loaded["scores"])
```

9.3 CSV

CSV looks simple but can contain quoted commas, missing values, and different line endings. Use the standard library rather than splitting on commas yourself.

```
1 import csv
2
3 with open("people.csv", newline="", encoding="utf-8") as file:
4     for row in csv.DictReader(file):
5         print(row["name"], row["age"])
```

9.4 Paths and directories

Build paths with `Path`, not by manually joining strings. A path is not the same thing as the contents of a file; it is a location.

```
1 from pathlib import Path
2
3 data_dir = Path("data")
4 data_dir.mkdir(exist_ok=True)
5 for path in data_dir.glob("*.json"):
6     print(path.name)
```

Syntax. Never build a shell command by inserting raw user input into a string. Validate paths and use safe process APIs. This principle becomes critical when programs handle uploads, filenames, or database queries.

Objects and classes

10.1 Objects bundle data and behaviour

An object combines state with operations that know how to work on that state. A class is a description used to create objects.

```
1 class BankAccount:
2     def __init__(self, owner, balance=0):
3         self.owner = owner
4         self.balance = balance
5
6     def deposit(self, amount):
7         if amount <= 0:
8             raise ValueError("amount must be positive")
9         self.balance += amount
10
11    def summary(self):
12        return f"{self.owner}: {self.balance:.2f}"
13
14 account = BankAccount("Ada", 100)
15 account.deposit(25)
16 print(account.summary())
```

`self` refers to the particular object receiving the method call. The constructor `__init__` runs when a new object is created.

10.2 Encapsulation and invariants

An **invariant** is a rule that should remain true, such as a bank balance not being allowed to become negative. Put the rule close to the state it protects. Public methods can reject invalid changes.

Python does not enforce private fields strongly, but a leading underscore communicates "internal use." In languages with access modifiers, fields can be declared private and exposed through methods or properties.

10.3 Inheritance and composition

Inheritance expresses an "is a" relationship. Composition expresses a "has a" relationship. Prefer composition when it avoids a brittle hierarchy.

```
1 class EmailSender:
2     def send(self, message):
3         print(f"Sending email: {message}")
```

```
4
5 class NotificationService:
6     def __init__(self, sender):
7         self.sender = sender
8
9     def notify(self, message):
10        self.sender.send(message)
11
12 service = NotificationService(EmailSender())
13 service.notify("Build finished")
```

The service depends on an object with a `send` operation. That dependency can later be replaced with a fake sender in a test.

10.4 Dataclasses

Use a dataclass when the main purpose of a class is to hold data.

```
1 from dataclasses import dataclass
2
3 @dataclass
4 class Point:
5     x: float
6     y: float
7
8 point = Point(3, 4)
9 print(point)
```

10.5 Type hints

Type hints document expectations and enable static analysis. Python does not normally enforce them at runtime.

```
1 def average(values: list[float]) -> float:
2     return sum(values) / len(values)
```

Hints are communication. They become especially valuable as a project grows or multiple people work on it.

Data structures and algorithms

11.1 Choosing a structure

The right structure depends on the operation you need most often.

Need	Typical choice
Ordered items, append at end	list / array
Fast membership and uniqueness	set / hash set
Named fields	dictionary / map / object
First-in, first-out work	queue
Last-in, first-out work	stack
Relationships between nodes	graph
Hierarchical relationships	tree

11.2 Complexity intuition

Time complexity describes how the amount of work grows as input size n grows. Common categories are:

- $O(1)$: constant time; the size of the input does not change the basic work.
- $O(\log n)$: logarithmic; each step discards a large part of the search space.
- $O(n)$: linear; visit each item once.
- $O(n \log n)$: common for efficient comparison sorting.
- $O(n^2)$: quadratic; compare many pairs or nest two full loops.

Big-O ignores constants and focuses on growth. It is a model, not a stopwatch.

11.3 Searching and sorting

Linear search is simple:

```
1 def find_index(items, target):
2     for index, item in enumerate(items):
3         if item == target:
4             return index
5     return -1
```

If a list is sorted, binary search can repeatedly halve the remaining range. Python provides `bisect`; do not implement a clever search until the simpler version is understood and measured.

Sorting with a key function is a powerful pattern:

```
1 people = [
2     {"name": "Ada", "age": 36},
3     {"name": "Alan", "age": 41},
4 ]
5 ordered = sorted(people, key=lambda person: person["age"])
```

11.4 Recursion

A recursive function calls itself on a smaller problem and needs a base case.

```
1 def factorial(n):
2     if n < 0:
3         raise ValueError("n must be non-negative")
4     if n == 0:
5         return 1
6     return n * factorial(n - 1)
```

Recursion is elegant for trees and divide-and-conquer algorithms. Iteration is often more memory-efficient for simple counting tasks.

11.5 Graphs

Represent a small graph with an adjacency dictionary:

```
1 graph = {
2     "A": ["B", "C"],
3     "B": ["D"],
4     "C": ["D"],
5     "D": [],
6 }
```

Breadth-first search uses a queue. Depth-first search uses a stack or recursion. These patterns appear in route finding, dependency resolution, web crawling, and game search.

The terminal, environments, and Git

12.1 The command line

A terminal lets you interact with the operating system by typing commands. The exact commands differ slightly between operating systems, but the underlying concepts are shared.

```
1 pwd                # show the current directory
2 ls                 # list files on macOS/Linux
3 cd project         # move into a directory
4 mkdir src          # create a directory
5 python --version    # show the Python version
```

On Windows PowerShell, `Get-ChildItem` is the long form of `ls`, but many common aliases work. The shell runs programs, connects them with pipes, redirects output, and exposes environment variables.

```
1 python app.py > output.txt
2 cat output.txt | grep error
```

12.2 Virtual environments

A project should not rely on a random global set of packages.

```
1 python -m venv .venv
2 source .venv/bin/activate    # macOS/Linux
3 .venv\Scripts\activate       # Windows PowerShell
4 python -m pip install requests
5 python -m pip freeze > requirements.txt
```

Keep the environment directory out of version control. Record the dependencies that another person needs to recreate the environment.

12.3 Git's mental model

Git records snapshots of a project. The working tree contains current files. The staging area is a proposed next snapshot. A commit is a named snapshot in history. A branch is a movable label pointing to a line of commits.


```
1 git init
2 git status
3 git add .
4 git commit -m "Add first program"
5 git log --oneline
6 git switch -c feature/cli
```

Commit small coherent changes. A good commit message says what changed, not how long you worked.

Remember. Version control is not only a backup. It is a communication system, an experiment log, and a way to make changes reversible.



The programming landscape

HTML: the structure of the web

13.1 Documents and elements

HTML describes the structure and meaning of a document. It is not primarily a programming language; it is a markup language. An element usually has an opening tag, content, and a closing tag.

```
1 <!doctype html>
2 <html lang="en">
3   <head>
4     <meta charset="utf-8">
5     <title>My first page</title>
6   </head>
7   <body>
8     <main>
9       <h1>Learn by making things</h1>
10      <p>HTML gives the page structure.</p>
11      <a href="https://example.com">Visit a site</a>
12    </main>
13  </body>
14 </html>
```

Use semantic elements such as `main`, `nav`, `article`, and `footer`. A **class** is a reusable styling hook. An **id** identifies one element and can be a target for links or scripts.

13.2 Forms

```
1 <form action="/search" method="get">
2   <label for="query">Search</label>
3   <input id="query" name="q" type="search" required>
4   <button type="submit">Search</button>
5 </form>
```

The `name` attribute is the key sent to the server. The label improves accessibility and makes the input easier to use.

13.3 Accessibility basics

Use headings in order, provide alternative text for meaningful images, label form controls, use buttons for actions, and do not communicate important information by colour alone. Semantic HTML helps browsers, search engines, assistive technologies, and future developers.

CSS: styling and layout

14.1 Selectors and declarations

CSS applies declarations to selected elements. A declaration has a property and a value.

```
1 :root {
2   --ink: #111827;
3   --accent: #0f766e;
4 }
5
6 body {
7   color: var(--ink);
8   font-family: system-ui, sans-serif;
9   line-height: 1.6;
10 }
11
12 .card {
13   border: 1px solid #e2e8f0;
14   border-radius: 12px;
15   padding: 1rem;
16 }
```

The dot selects a class. A hash selects an id. A bare element name selects all elements of that type. CSS has a cascade: more specific or later rules can override earlier rules.

14.2 The box model

Every element has content, padding, border, and margin. A reliable default is:

```
1 * {
2   box-sizing: border-box;
3 }
```

This makes declared width include padding and border, which is usually easier to reason about.

14.3 Flexbox and grid

Flexbox is useful for laying items along one dimension. Grid is useful for rows and columns.

```
1 .toolbar {
2   display: flex;
3   align-items: center;
```

```
4   justify-content: space-between;
5   gap: 1rem;
6 }
7
8 .layout {
9   display: grid;
10  grid-template-columns: 240px 1fr;
11  gap: 2rem;
12 }
```

Start with normal document flow. Add positioning only when you can explain which containing block the element uses.

JavaScript: behaviour in the browser

15.1 Variables and values

JavaScript has primitive values such as strings, numbers, booleans, `null`, and `undefined`, plus objects such as arrays and dictionaries.

```
1  const name = "Ada";
2  let count = 0;
3  count += 1;
4
5  const skills = ["math", "code"];
6  const user = { name, skills, active: true };
7  console.log(user.name);
```

Prefer `const` by default. Use `let` when a binding must change. Avoid `var` in new code.

15.2 Functions and conditions

```
1  function greet(person) {
2    return `Hello, ${person}!`;
3  }
4
5  const isAdult = age => age >= 18;
6
7  if (isAdult(26)) {
8    console.log(greet("Ada"));
9  } else {
10   console.log("Not an adult");
11 }
```

Arrow functions capture the surrounding `this` value differently from ordinary functions. Learn the distinction before using methods that depend on `this`.

15.3 Arrays and objects

```
1  const numbers = [1, 2, 3, 4];
2  const squares = numbers.map(number => number * number);
3  const even = numbers.filter(number => number % 2 === 0);
4  const total = numbers.reduce((sum, number) => sum + number, 0);
```

The triple equals operator compares without the coercion rules of `==`. In modern code, use `===` and `!==` unless you have a specific reason not to.

15.4 The DOM and events

```
1  const button = document.querySelector("#save");
2  const output = document.querySelector("#output");
3
4  button.addEventListener("click", () => {
5    output.textContent = "Saved";
6  });
```

The browser exposes the document as an object tree. An event listener runs a function later when an event occurs. This is the beginning of event-driven programming.

TypeScript: JavaScript with a type system

16.1 Annotations and inference

TypeScript adds static types and then compiles to JavaScript.

```
1 let count: number = 0;
2 const name: string = "Ada";
3 const active: boolean = true;
4
5 function greet(person: string): string {
6   return `Hello, ${person}!`;
7 }
```

TypeScript often infers obvious types. Do not annotate every local variable just to make the file noisy. Add types at boundaries: function parameters, return values, API data, and public interfaces.

16.2 Objects, interfaces, and unions

```
1 interface User {
2   id: number;
3   name: string;
4   role?: "admin" | "member";
5 }
6
7 function label(user: User): string {
8   const role = user.role ?? "member";
9   return `${user.name} (${role})`;
10 }
```

The question mark makes a property optional. The vertical bar creates a union: the value must be one of the allowed alternatives.

16.3 Generics

Generics let a function preserve the relationship between its input and output.

```
1 function first<T>(items: T[]): T | undefined {
2   return items[0];
3 }
```



```
4
5 const firstName = first(["Ada", "Grace"]);
6 const firstNumber = first([10, 20]);
```

16.4 Narrowing

```
1 function format(value: string | number): string {
2   if (typeof value === "number") {
3     return value.toFixed(2);
4   }
5   return value.toUpperCase();
6 }
```

The compiler narrows the union after the condition. A type assertion such as `value as User` tells the compiler to trust you; use it only when you can justify it.

SQL and databases

17.1 Tables and relationships

A relational database stores rows in tables. A row is one record. A column is an attribute. A primary key identifies a row. A foreign key connects a row to another table.

```
1 CREATE TABLE users (  
2     id INTEGER PRIMARY KEY,  
3     name TEXT NOT NULL,  
4     email TEXT UNIQUE NOT NULL  
5 );  
6  
7 CREATE TABLE posts (  
8     id INTEGER PRIMARY KEY,  
9     user_id INTEGER NOT NULL REFERENCES users(id),  
10    title TEXT NOT NULL,  
11    created_at TIMESTAMP NOT NULL  
12 );
```

17.2 Selecting data

```
1 SELECT name, email  
2 FROM users  
3 WHERE email LIKE '%example.com'  
4 ORDER BY name ASC;
```

The order is conceptually FROM, WHERE, SELECT, and ORDER BY, although the written syntax has a different order. SQL describes what result you want; the database chooses a plan to obtain it.

17.3 Inserting and updating

```
1 INSERT INTO users (name, email)  
2 VALUES ('Ada', 'ada@example.com');  
3  
4 UPDATE users  
5 SET name = 'Ada Lovelace'  
6 WHERE email = 'ada@example.com';
```

The WHERE clause is essential for safe updates. Without it, every row may be modified.

17.4 Joins and aggregation

```
1 SELECT users.name, COUNT(posts.id) AS post_count
2 FROM users
3 LEFT JOIN posts ON posts.user_id = users.id
4 GROUP BY users.id, users.name
5 HAVING COUNT(posts.id) > 0
6 ORDER BY post_count DESC;
```

Joins combine related tables. **Aggregation** reduces many rows to summaries. A `LEFT JOIN` keeps users even when they have no posts; an inner `JOIN` would discard those users.

17.5 Parameterized queries

Never build SQL by concatenating user input. Use parameters supplied by your database driver.

```
1 cursor.execute(
2     "SELECT id, name FROM users WHERE email = ?",
3     (email,)
4 )
```

The placeholder syntax differs by driver. The principle is the same: keep code and data separate.

HTTP, APIs, JSON, and asynchronous code

18.1 Requests and responses

HTTP is a request-response protocol. A request has a method, URL, headers, and sometimes a body. A response has a status code, headers, and a body.

Method	Common meaning
GET	Read a resource
POST	Create or trigger an action
PUT / PATCH	Replace or partially update a resource
DELETE	Remove a resource

Status codes communicate broad outcomes: 2xx success, 3xx redirection, 4xx client-side problem, and 5xx server-side problem.

18.2 A JSON API client

```
1 import requests
2
3 response = requests.get(
4     "https://api.example.com/users",
5     timeout=10,
6 )
7 response.raise_for_status()
8 data = response.json()
9 for user in data["users"]:
10     print(user["name"])
```

Set a timeout. Check the status. Treat the response shape as external data and validate it before trusting nested keys.

18.3 Async and await

Asynchronous code lets a program start an operation that may take time, such as a network request, while allowing other work to progress.

```
1 import asyncio
2
3 async def fetch_value(delay, value):
4     await asyncio.sleep(delay)
```

```
5     return value
6
7  async def main():
8      first, second = await asyncio.gather(
9          fetch_value(1, "A"),
10         fetch_value(1, "B"),
11     )
12     print(first, second)
13
14  asyncio.run(main())
```

`await` pauses the current coroutine until an awaitable completes. It does not automatically make CPU-heavy work faster. Async code is most useful when the program spends time waiting for external operations.

18.4 Promises in JavaScript

```
1  async function loadUsers() {
2      const response = await fetch("/api/users");
3      if (!response.ok) {
4          throw new Error(`HTTP ${response.status}`);
5      }
6      return response.json();
7  }
8
9  loadUsers()
10     .then(users => console.log(users))
11     .catch(error => console.error(error));
```

An async function returns a promise. Handle failure explicitly; a network request can fail even when the code is correct.

Backend programming

19.1 The server's job

A backend receives requests, authenticates and validates them, applies business rules, reads or writes data, and returns a response. A framework supplies routing, middleware, serialization, and integration points.

```
1 from fastapi import FastAPI
2
3 app = FastAPI()
4
5 @app.get("/hello")
6 def hello(name: str = "world"):
7     return {"message": f"Hello, {name}!"}
```

The decorator registers a route. The function is the handler. The returned dictionary is serialized to JSON by the framework.

19.2 Layers

A small application can be divided into:

1. Transport: HTTP, routing, headers, status codes.
2. Application: use cases and orchestration.
3. Domain: rules and concepts that matter to the product.
4. Persistence: SQL, files, caches, external services.

The point is not to create folders for their own sake. The point is to keep changes local and make dependencies visible.

19.3 Authentication and authorization

Authentication asks "who are you?" Authorization asks "what are you allowed to do?" Check both on the server. Never trust a role or user id supplied by the browser merely because the interface hides a button.

19.4 Validation

Validate at boundaries. Reject invalid data early with a useful error. Keep internal functions simpler by giving them well-formed values. Make validation rules explicit and test them.

C and C++: closer to the machine

20.1 A small C++ program

```

1  #include <iostream>
2  #include <string>
3
4  int main() {
5      std::string name = "Ada";
6      std::cout << "Hello, " << name << "!\n";
7      return 0;
8  }
```

The preprocessor includes declarations. The compiler checks and translates the source. The linker combines compiled pieces into an executable. C++ uses static types and gives you direct control over memory and object lifetimes.

20.2 References and pointers

A pointer stores an address. A reference is another name for an existing object.

```

1  void add_one(int& value) {
2      value += 1;
3  }
4
5  int number = 4;
6  add_one(number);
```

The ampersand in a parameter makes `value` a reference, so the function changes the caller's variable. Modern C++ often uses standard library containers and smart pointers rather than manual memory management.

20.3 Vectors, maps, and loops

```

1  #include <map>
2  #include <vector>
3
4  std::vector<int> numbers{1, 2, 3};
5  for (int number : numbers) {
6      std::cout << number << "\n";
7  }
8
9  std::map<std::string, int> scores{{"Ada", 97}};
```

The syntax is more explicit than Python's, but the ideas are the same: values, variables, collections, loops, functions, and objects.

20.4 Memory as a model

A variable may live in automatic storage, dynamic storage, or inside another object. Bugs such as use-after-free, buffer overflow, and data races arise when the program's assumptions about lifetime or access are wrong. These details are why systems programming deserves dedicated study.

Java and the JVM

21.1 Classes and a main method

```
1 public class Hello {  
2     public static void main(String[] args) {  
3         String name = "Ada";  
4         System.out.println("Hello, " + name + "!");  
5     }  
6 }
```

Java source is compiled to bytecode and executed by the Java Virtual Machine. The JVM provides portability, garbage collection, a large standard library, and mature tooling.

21.2 Types and collections

```
1 import java.util.List;  
2  
3 List<Integer> numbers = List.of(1, 2, 3);  
4 for (int number : numbers) {  
5     System.out.println(number * number);  
6 }
```

Primitive types such as `int` differ from reference types such as `String`. Generic types describe what a collection contains. Java's syntax is verbose, but its explicitness can make large codebases easier to analyse.

21.3 Exceptions and interfaces

```
1 interface Sender {  
2     void send(String message);  
3 }  
4  
5 class ConsoleSender implements Sender {  
6     public void send(String message) {  
7         System.out.println(message);  
8     }  
9 }
```

An interface describes a contract. A class implements the contract. This is the same dependency-inversion idea seen in the Python composition example.



Beyond syntax

Programming paradigms and architecture

22.1 Imperative and declarative code

Imperative code describes steps: set a variable, loop, call a function. Declarative code describes a desired result: a SQL query, an HTML document, or a CSS layout. Most real systems mix both styles.

22.2 Procedural programming

Procedural code organises a task into functions and sequences. It is often the best starting point because the control flow is visible.

22.3 Object-oriented programming

Object-oriented code organises behaviour around objects and their relationships. The useful parts are encapsulation, polymorphism, and boundaries. Inheritance is only one mechanism and is often less important than composition.

22.4 Functional programming

Functional programming treats functions as values and prefers transformations that do not mutate shared state.

```
1 numbers = [1, 2, 3, 4]
2 result = list(map(lambda n: n * 2, numbers))
```

Pure functions are easier to test because the output depends only on the input. Immutability reduces the number of places that can unexpectedly change state.

22.5 Architecture as boundaries

Architecture is the arrangement of responsibilities and dependencies. A useful design question is: if this requirement changes, how many unrelated files must change? Good boundaries reduce that number.

Common styles include layered architecture, hexagonal architecture, event-driven systems, client-server applications, and pipelines. Treat these as tools, not identities.

Concurrency, parallelism, and processes

23.1 Concurrency is not the same as parallelism

Concurrency means managing multiple tasks whose progress overlaps. **Parallelism** means doing multiple computations at the same time, usually on different CPU cores. A program can be concurrent without being parallel.

23.2 Processes and threads

A process has its own address space and resources. Threads share memory inside a process. Shared memory is fast, but it introduces races: the result can depend on timing.

```
1 from concurrent.futures import ThreadPoolExecutor
2
3 def square(number):
4     return number * number
5
6 with ThreadPoolExecutor(max_workers=4) as pool:
7     results = list(pool.map(square, range(6)))
8 print(results)
```

Use threads mainly for I/O-bound work in Python. Use processes or native extensions for CPU-heavy work, remembering that process communication has a cost.

23.3 Shared state and locks

If two workers can read and write the same state, protect the critical section with a lock or use a design that avoids shared mutation.

```
1 from threading import Lock
2
3 balance = 0
4 lock = Lock()
5
6 def deposit(amount):
7     global balance
8     with lock:
9         balance += amount
```

Locks prevent certain races but can create deadlocks if acquired in inconsistent orders. Prefer

simple ownership and message passing when possible.

Security and defensive programming

24.1 Threat modelling

Security starts with asking what needs protection, who might attack it, what capabilities they have, and what the impact would be. A login page is not a complete security model.

24.2 The boundary rule

Treat data from browsers, files, APIs, cookies, headers, environment variables, and databases as untrusted until validated. Encode output for its destination. SQL needs parameters. HTML needs escaping or a safe templating system. Shell commands need safe argument APIs.

24.3 Secrets

Do not commit passwords, API keys, private certificates, or tokens. Store secrets in a secret manager or environment configuration, rotate them when exposed, and give each service the minimum permissions it needs.

24.4 Common failures

Failure	Better habit
SQL injection	Parameterized queries
Cross-site scripting	Context-aware output escaping
Broken access control	Authorize every protected operation on the server
Leaked secrets	Environment or secret-manager configuration
Insecure file upload	Validate type, size, name, and storage location
Weak authentication	Use a mature identity system and secure password hashing

Security is a property of the whole system, not a single "security function."

Performance, deployment, and cloud systems

25.1 Measure before optimising

Performance work begins with a user-visible goal: response time, throughput, memory use, startup time, or cost. Profile the real bottleneck. Faster code in the wrong place changes nothing.

25.2 Caching

A cache stores a result so the next request can avoid expensive work. Every cache needs an invalidation strategy, a freshness rule, and a fallback for misses. A cache can reduce load while also making correctness harder.

25.3 Build and deployment

A deployment pipeline commonly:

1. checks formatting and types;
2. runs unit and integration tests;
3. builds a versioned artifact or container;
4. deploys to a staging environment;
5. runs smoke tests;
6. releases gradually and monitors the result.

25.4 Containers

A container packages an application and its dependencies in a reproducible image. It still shares the host kernel and is not a complete security boundary by itself.

```
1 docker build -t my-app .  
2 docker run --rm -p 8000:8000 my-app
```

25.5 Observability

Logs describe events. Metrics describe quantities over time. Traces connect work across services. Good observability answers: what failed, for whom, when, how often, and what changed?

Data science and machine learning syntax

26.1 Arrays and tables

Numerical work often uses arrays rather than ordinary Python lists. A dataframe represents labelled rows and columns.

```
1 import numpy as np
2 import pandas as pd
3
4 values = np.array([1, 2, 3, 4])
5 scaled = values * 10
6
7 frame = pd.DataFrame({
8     "name": ["Ada", "Grace"],
9     "score": [97, 95],
10 })
11 print(frame[frame["score"] > 95])
```

The expression `values * 10` applies the operation element by element. This is called vectorisation and is usually faster and clearer than a Python loop for numerical arrays.

26.2 A minimal learning loop

```
1 for epoch in range(100):
2     prediction = model(x)
3     loss = loss_function(prediction, y)
4     optimizer.zero_grad()
5     loss.backward()
6     optimizer.step()
```

The model makes a prediction, compares it with a target, computes a loss, calculates gradients, and updates parameters. The syntax changes between libraries; the loop's structure is widely shared.

26.3 Data leakage and evaluation

Split data into training and evaluation sets before tuning decisions. Do not let information from the evaluation set influence the training process. Report the metric that matches the real decision, not only the metric that looks impressive.

How computers and networks work

27.1 Bits, bytes, and memory

A bit is 0 or 1. Eight bits make a byte. Text, images, numbers, and programs are all encoded as bytes. The same bytes can represent different values depending on the encoding and interpretation.

The CPU executes instructions using registers and caches. RAM stores working data. Persistent storage keeps data after power is removed. The operating system manages processes, files, devices, and permissions.

27.2 Call stacks and heaps

The call stack tracks active function calls and local variables. The heap stores dynamically allocated objects. A stack frame disappears when a function returns; heap objects live until a memory manager or explicit code releases them.

27.3 Networks

An IP address identifies a network endpoint. DNS maps names to addresses. TCP provides an ordered reliable byte stream. TLS encrypts and authenticates a connection. HTTP defines a common application-level request-response format.

The layers are abstractions. A browser can issue an HTTP request without exposing every packet, checksum, route, or radio signal to the programmer. When debugging, move down a layer only when the current layer cannot explain the problem.

27.4 The request path

When a browser loads a page, a simplified path is:

1. resolve the domain through DNS;
2. establish a network connection and, for HTTPS, a secure session;
3. send an HTTP request;
4. route the request through a server and application;
5. read data, perhaps from a database or cache;
6. return bytes that the browser parses into a document and pixels.

This path explains why a page can be slow even when a small function is fast: waiting, distance, queueing, database work, rendering, and many other components matter.

Projects and a path from zero to capable

28.1 The smallest useful projects

Build projects that force you to use input, state, functions, files, errors, and tests. Good progression:

1. A command-line calculator with validation.
2. A text-based guessing game with a score.
3. A file-backed to-do list using JSON.
4. A web page with a form and client-side JavaScript.
5. A small API that stores records in SQLite.
6. A dashboard that reads an API and renders a table.
7. A project with authentication, tests, deployment, and monitoring.

28.2 A project checklist

Before writing code, write a one-paragraph description. Define the inputs, outputs, invalid cases, and one example. Then make the smallest vertical slice work end to end. Add one feature at a time.

Try it. Design a to-do application. Write down its data model, three commands, two invalid cases, and one test for each command. Implement the command-line version before adding a web interface.

28.3 A twelve-week route

Weeks	Focus
1-2	Python values, expressions, strings, conditions, loops
3-4	Collections, functions, files, exceptions, testing
5	Git, terminal, environments, small CLI project
6-7	HTML, CSS, JavaScript, browser events
8	SQL, tables, joins, parameterized queries
9	HTTP, JSON, APIs, asynchronous code
10	Backend routes, validation, authentication concepts
11	Data structures, complexity, algorithms, profiling
12	Finish, document, test, deploy, and explain one project

Do not measure progress by how many tutorials you have watched. Measure it by whether you can make a small change, explain why it works, and debug it when it fails.

Core idea. You are "good at programming" when you can move from an unclear problem to a tested, understandable solution. Syntax is the alphabet. Problem decomposition is the language.

Syntax atlas

This appendix is a compact memory aid. Use it after understanding the ideas in the chapters. A cheat sheet cannot replace practice, but it can reduce the friction of recalling a shape.

A.1 Python

Shape	Purpose
<code>x = 3</code>	assignment
<code>if condition:</code>	conditional block
<code>for item in items:</code>	iteration
<code>while condition:</code>	repeated condition
<code>def name(arg):</code>	function definition
<code>return value</code>	return from a function
<code>items.append(x)</code>	mutate a list
<code>items[1:4]</code>	slice a sequence
<code>key in mapping</code>	membership test
<code>try: ... except Error:</code>	handle an exception
<code>with open(...) as f:</code>	managed resource
<code>class Name:</code>	define a class
<code>import package</code>	import a module

A.2 JavaScript and TypeScript

Shape	Purpose
<code>const x = 3;</code>	stable binding
<code>let x = 3;</code>	reassignable binding
<code>function f(x) \{ return x; \}</code>	function
<code>const f = x => x * 2;</code>	arrow function
<code>if (ok) \{ ... \}</code>	conditional block
<code>for (const x of xs) \{ ... \}</code>	iteration
<code>xs.map(x => x * 2)</code>	transform an array
<code>await fetch(url)</code>	await an async result
<code>interface User \{ ... \}</code>	TypeScript shape
<code>type Result = A B</code>	union type

A.3 SQL

Shape	Purpose
SELECT columns FROM table	read rows
WHERE condition	filter rows
ORDER BY column DESC	sort results
INSERT INTO table (...) VALUES (...)	create a row
UPDATE table SET ... WHERE ...	change rows
DELETE FROM table WHERE ...	remove rows
JOIN other ON relation	combine tables
GROUP BY column	aggregate groups
CREATE TABLE ...	define a table

A.4 HTML, CSS, and Bash

Language	Essential shapes
HTML	<code><tag attr="value">content</tag></code>
CSS	<code>selector {\ property: value; }</code>
Bash	<code>command command, VAR=value, if ...; then ...; fi</code>
C++	<code>type name = value;; if (...) {\ ... }, for (type x : xs)</code>
Java	<code>class Name {\ ... }, public static void main(...)</code>

Glossary

API	A defined interface through which software components communicate.
Argument	A value supplied to a function call.
Array	An ordered, indexable collection, often stored contiguously.
Assignment	Binding a name or location to a value.
Boolean	A value with two logical states, true and false.
Bug	A defect that causes a program to behave incorrectly.
Class	A description from which objects can be created.
Compiler	A program that translates source into another executable form.
Database	A system for storing, querying, and updating structured data.
Exception	A signal that normal execution cannot continue at the current point.
Function	A named or anonymous unit of reusable behaviour.
Hash table	A structure that maps keys to values using a hash function.
HTTP	A protocol used by web clients and servers.
Immutable	Not changeable after creation.
Interpreter	A runtime that executes a program representation.
Iteration	Repeating a process over items or while a condition holds.
Library	Reusable code packaged for use by other programs.
Method	A function associated with an object or class.
Module	A file or unit of code that can be imported.
Object	A value that can bundle state and behaviour.
Parameter	A name in a function definition that receives an argument.
Parse	Convert source text into a structured representation.
Process	A running program with its own operating-system resources.
Recursion	Defining a process in terms of a smaller version of itself.
Runtime	The environment in which a program executes.
Scope	The region in which a name can be used.

Glossary

Server	A program that provides a service to clients.
State	The data that describes a system at a particular time.
Syntax	The formal grammar of a language.
Type	A category of values and the operations valid for them.
Variable	A name whose associated value may change.

Selected practice solutions

The point of an exercise is to attempt it before reading a solution. These examples show one reasonable approach, not the only correct approach.

C.1 A number range

```
1 def in_range(number):  
2     return 10 <= number <= 20
```

C.2 Word lengths

```
1 sentence = "learning code takes repeated practice"  
2 words = sentence.split()  
3 long_lengths = [len(word) for word in words if len(word) > 4]  
4 length_by_word = {word: len(word) for word in words}
```

C.3 A small command-line to-do model

```
1 def add_task(tasks, title):  
2     tasks.append({"title": title, "done": False})  
3  
4 def complete_task(tasks, index):  
5     tasks[index]["done"] = True  
6  
7 tasks = []  
8 add_task(tasks, "Learn functions")  
9 complete_task(tasks, 0)  
10 print(tasks)
```

Extend this model with validation, JSON persistence, a command-line parser, and tests. Each extension is a chance to practise a different layer of the book.

Final checklist

Before calling a project finished, ask:

- Can a new person run it from a clean environment?
- Are the main inputs and outputs documented?
- Are invalid cases handled deliberately?
- Are the core behaviours covered by tests?
- Are secrets and personal data protected?
- Can I explain the data model and the most important dependency?
- Have I measured the part I claim is slow?
- Is the code easier to change after my last refactor, or merely shorter?

Keep building. Keep explaining.

The fastest route from zero is a small project finished carefully.

Index

Aggregation, 35

class, 27

compiler, 2

Concurrency, 44

expression, 6

id, 27

interpreter, 2

invariant, 20

Joins, 35

logic error, 16

Parallelism, 44

runtime error, 16

Semantics, 2

short-circuit evaluation, 7

state, 2

Syntax, 2

syntax error, 16

Time complexity, 22

type, 4

value, 4

virtual machine, 2